JPMorgan Chase — Lead Site Reliability Engineer

Interview Questions & Production-Ready Spoken Answers

Role focus: Infrastructure Platforms • SRE leadership • Production coding • Declarative systems • SLOs • Observability • Incident response • Kubernetes/Terraform • AI-native SRE • Security & operational risk

How to use this guide: Answers are deliberately written in a natural spoken style. Replace any example that is not part of your actual experience. For scenario questions, use: **Impact** → **Triage** → **Evidence** → **Isolate** → **Mitigate** → **Recover** → **RCA** → **Prevent**.

1. Opening / Leadership

Q. Tell me about yourself and why this Lead SRE role fits you.

Spoken answer: I'm an Azure-certified SRE/DevOps engineer with 11+ years of experience across cloud, Kubernetes, CI/CD, infrastructure automation, observability and production operations. In my recent Azure Synapse Analytics project, I supported globally distributed customers, handled P1/P2 incidents, worked with SLI/SLOs, reliability dashboards, alert optimization, capacity planning and deployment automation. My strength is combining hands-on troubleshooting with reliability engineering: I look at the failure mode, automate repetitive work, improve telemetry and then put a durable control in place. This role fits because it expects both technical depth and leadership—owning reliability outcomes, decomposing ambiguous problems, mentoring engineers and building self-healing systems rather than relying on manual operations.

Q. How do you operate as a technical lead without becoming a bottleneck?

Spoken answer: I establish the outcome, constraints and reliability guardrails first, then decompose the problem into independently executable work. I assign ownership based on strengths, define interfaces and acceptance criteria, and use design reviews and lightweight checkpoints rather than approving every implementation detail. For high-risk changes I insist on testing, observability, rollback and operational readiness. My goal as a lead is to increase the team's decision quality and throughput, not centralize decisions around myself.

Q. How would you approach a resiliency design review?

Spoken answer: I start with business-critical user journeys and dependencies, then map failure domains, interfaces, state, recovery paths and blast radius. I look at availability targets, RTO/RPO where applicable, capacity limits, dependency failure, data corruption, deployment failure and regional or zonal failure. I verify that telemetry can detect the failure and that automation can safely mitigate it. Finally, I check whether the design has tested recovery procedures, clear ownership and measurable SLOs. I want the review to produce concrete engineering actions, not just a document.

Q. How do you mentor engineers on reliability?

Spoken answer: I use production problems as learning opportunities. I explain the reasoning behind triage, observability, failure isolation and safe mitigation rather than simply giving commands. For design work, I ask engineers to articulate failure modes, SLO impact and rollback strategy. I also encourage reusable automation and runbooks. The outcome I want is an engineer who can independently diagnose the next incident and improve the system so that the same class of failure becomes less likely.

2. SRE Fundamentals / SLOs

Q. Explain SLI, SLO, SLA and error budget.

Spoken answer: An SLI is the measured indicator of service behavior—for example successful request rate or latency. An SLO is the internal reliability target for that indicator, such as 99.9% successful requests. An SLA is a customer or contractual commitment that may include consequences if it is missed. The error budget is the amount of unreliability allowed by the SLO. I use the budget to balance reliability and delivery: if the service is consuming too much budget, we prioritize reliability work; if it has healthy budget, we can take calculated delivery risk.

Q. How would you define SLOs for an enterprise platform?

Spoken answer: I begin with user journeys rather than infrastructure metrics. For an API, availability and latency may be primary; for a data platform, successful job completion, freshness and processing latency may matter. I define the measurement window, exclusions and aggregation carefully so the SLO reflects user experience. Then I validate the target with stakeholders, instrument the telemetry, build dashboards and alerts around error-budget consumption, and review the SLO periodically as the service evolves.

Q. What is SLO-based alerting and why is it better than CPU alerts?

Spoken answer: CPU can be useful diagnostic telemetry, but it is not necessarily a user-impact signal. SLO-based alerting focuses on whether the service is burning reliability budget too quickly. For example, a fast burn alert can indicate that an outage or severe degradation will consume the error budget rapidly. I still use infrastructure metrics for diagnosis, but the paging signal should generally correlate with meaningful service impact.

Q. What would you do if stakeholders demand 99.99% availability?

Spoken answer: I would first translate 99.99% into an actual error budget and determine whether the architecture and dependencies can realistically support it. I would identify the critical path, failure domains, dependency SLOs, recovery time and capacity requirements. Then I would quantify the engineering and operational cost of moving from the current target to 99.99%. The decision should be based on business impact, technical feasibility and cost rather than simply choosing a more impressive number.

Q. How do you measure and reduce toil?

Spoken answer: I establish a baseline: hours per week, frequency, engineer count, failure recurrence and manual steps. Then I classify the work—alert noise, repetitive remediation, deployment tasks, data collection or operational reporting. I automate the highest-volume and highest-risk tasks first, preferably through idempotent tooling and closed-loop controls. After deployment, I measure the reduction in manual effort and incident recurrence. If an automation does not measurably reduce toil or risk, I reconsider it.

3. Python / Production Coding

Q. This is a software engineering SRE role. How strong are you in Python?

Spoken answer: I use Python for operational automation, API integrations, validation, data processing and reliability tooling. My approach is production-oriented: clear interfaces, structured logging, exception handling, retries with backoff where appropriate, timeouts, configuration management, tests and safe failure behavior. For automation that changes production state, I also make operations idempotent and add dry-run or validation paths when possible.

Q. Write a Python approach to detect services that exceed an error threshold.

Spoken answer: I would collect a bounded time window of request totals and errors, calculate the error ratio, compare it against a configured threshold, and emit a structured result. In production I would avoid a simplistic single-sample decision: I would use a rolling window, define minimum traffic, handle missing data explicitly, add unit tests, and integrate the result with the monitoring or alerting system. If remediation is triggered, the code should validate the target and make the action idempotent.

Q. How do you make automation production-safe?

Spoken answer: I use least privilege, input validation, timeouts, bounded retries, idempotency, structured logs and explicit failure handling. For destructive actions, I prefer approval gates or policy checks. I test against representative failure cases, use canary or staged rollout where appropriate, and ensure every automated action is auditable. The automation should fail safely rather than making an uncertain situation worse.

Q. What is the difference between a script and a reliability control loop?

Spoken answer: A script normally executes a sequence of imperative steps and stops. A control loop continuously compares observed state with desired state and reconciles the difference. The second model is stronger for reliability because it can detect drift and recover repeatedly. Kubernetes controllers are a good example: the desired state is declared, the controller observes reality and takes actions until the system converges.

4. Declarative / Self-Healing Systems

Q. Explain intent-based or declarative infrastructure.

Spoken answer: Instead of telling the system exactly how to make every change, I define the desired state as data—for example the required replicas, configuration or policy. A controller continuously observes actual state and reconciles it with the desired state. This makes the system repeatable, auditable and capable of correcting drift. Terraform and Kubernetes both demonstrate declarative principles, although their reconciliation models and state lifecycles differ.

Q. How would you design a self-healing service?

Spoken answer: I would first define what healthy means using measurable signals. Then I would identify failure modes that are safe to remediate automatically—for example restarting an unhealthy stateless workload or replacing a failed instance. The controller needs detection, decision logic, bounded retries, cooldowns, safeguards and observability. I would never automate an ambiguous or potentially destructive action without guardrails. Every remediation should be logged so we can determine whether it helped or masked a deeper issue.

Q. How do you prevent self-healing from becoming a failure amplifier?

Spoken answer: I use rate limits, cooldown periods, circuit breakers, maximum retry counts and health validation. I also distinguish between transient and systemic failures. If multiple instances fail simultaneously, restarting everything may amplify the outage. In that case the controller should stop escalating and surface the condition for human intervention. Automation needs an escape hatch and should preserve evidence for RCA.

Q. How do Terraform and Kubernetes differ operationally?

Spoken answer: Terraform is primarily used to provision and manage infrastructure and configuration through a declarative plan/apply workflow with state. Kubernetes continuously reconciles application and cluster desired state through controllers. Terraform is useful for creating the platform and its dependencies; Kubernetes handles continuous workload orchestration. I am careful about ownership boundaries so two systems do not continuously fight over the same resource.

5. Observability / Telemetry

Q. What does good observability mean to you?

Spoken answer: Good observability lets an engineer explain why a system is behaving the way it is from externally visible outputs and internal telemetry. I use metrics, logs and traces together, with correlation IDs and consistent dimensions. Dashboards should show user impact, service health and dependency health. Alerts should be actionable and tied to a meaningful condition. For major incidents, I want enough telemetry to move quickly from symptom to affected component and failure mode.

Q. White-box vs black-box monitoring?

Spoken answer: Black-box monitoring tests the system from the outside, such as synthetic HTTP checks, latency and availability. White-box monitoring uses internal telemetry such as request rate, error rate, queue depth, saturation and application metrics. I use both: black-box tells me whether users can successfully use the service; white-box helps explain why it is failing.

Q. How have you used Prometheus and Grafana?

Spoken answer: I've used Prometheus for metric collection and querying and Grafana for dashboards and visualization. I focus on useful dimensions rather than creating high-cardinality metrics unnecessarily. For an SRE dashboard I would show traffic, errors, latency, saturation, SLO status and dependency health, with drill-down paths into logs and traces. Alerting should be separated from dashboards: the alert needs a clear action, while the dashboard supports diagnosis.

Q. How do you reduce alert fatigue?

Spoken answer: I inventory alerts and classify them by actionability, frequency and user impact. Alerts that repeatedly fire without an action are candidates for threshold changes, aggregation, suppression or removal. I prefer symptom-based and SLO-based paging for user impact, while lower-severity diagnostic signals can go to dashboards or tickets. After every significant incident, I review whether the alert fired at the right time and whether it helped the responder.

6. Kubernetes / Cloud / Infrastructure

Q. A Kubernetes deployment is in CrashLoopBackOff. Walk me through your triage.

Spoken answer: I start with impact and scope: one pod, one deployment or a broader platform issue. Then I check pod status and events, describe the pod, inspect current and previous container logs, and check resource requests/limits. I verify configuration, Secrets, ConfigMaps, probes and recent changes. I then inspect node health and dependencies such as databases or external services. Once I identify the failure mode, I mitigate safely, restore service and then perform RCA. I avoid repeatedly restarting pods without understanding the cause.

Q. A pod is Pending. What do you check?

Spoken answer: I check describe output and scheduler events first. Common causes are insufficient CPU or memory, taints/tolerations, node selectors or affinity, unavailable topology constraints, PVC binding and quota. I compare the pod's requests against node allocatable capacity and check whether a required node label or storage class exists. The scheduler event normally narrows the cause quickly.

Q. How would you troubleshoot a production latency increase?

Spoken answer: I first establish when it started and whether it affects all traffic or a specific endpoint, region or customer segment. I compare request rate, error rate and latency percentiles with deployment and infrastructure changes. Then I use traces and application metrics to identify whether time is spent in the application, database, network or downstream dependency. I mitigate first if customer impact is significant—rollback, traffic shift, scaling or dependency isolation as appropriate—then investigate the root cause.

Q. How do you approach capacity planning?

Spoken answer: I use historical utilization and growth trends, but I also model saturation and failure behavior. I look at CPU, memory, network, storage, request rate, queue depth and latency under peak conditions. I define headroom based on the service's scaling time and risk tolerance. I also test capacity assumptions through load testing or controlled experiments. Capacity planning should result in an actionable threshold and scaling strategy, not just a monthly graph.

7. Incident Leadership

Q. Describe your approach to a P1 incident.

Spoken answer: My first priority is restoring customer impact safely. I establish an incident commander, identify the affected service and blast radius, and create a clear communication channel. I use metrics, logs, traces and recent changes to form hypotheses and test them quickly. We choose the lowest-risk mitigation that can restore service, such as rollback, traffic shift, scaling or disabling a faulty feature. Once stable, I preserve evidence, identify the root cause, document the timeline and implement corrective actions. The review is blameless and focused on system improvements.

Q. What if engineers disagree during an incident?

Spoken answer: I bring the discussion back to observable evidence and customer impact. Each hypothesis should have a test that can be performed quickly and safely. If two actions are possible, I prefer the reversible lower-risk action while collecting more evidence. I also make ownership explicit so multiple people do not make conflicting production changes. The incident commander owns the decision process; subject-matter experts provide evidence and recommendations.

Q. What makes a good post-incident review?

Spoken answer: It should reconstruct what happened, why the system allowed it, how detection and response worked, and which controls failed. I include the timeline, customer impact, contributing factors, detection gaps, mitigation and recovery. Actions should be specific, owned and measurable. A strong postmortem changes the system—automation, architecture, tests, alerts, capacity or runbooks—rather than simply documenting that people should be more careful.

Q. How do you handle a recurring incident?

Spoken answer: I treat recurrence as evidence that the previous corrective action was incomplete. I compare incidents to identify the common failure mode, then prioritize a durable control. That might be eliminating the dependency, adding automated recovery, improving validation, changing an SLO alert or redesigning a component. I also measure recurrence rate after the change to verify that the fix actually worked.

8. AI-Native SRE

Q. How would you use AI in the SRE lifecycle?

Spoken answer: I would use AI for high-volume cognitive work while keeping humans accountable for production decisions. Examples include summarizing incident telemetry, generating candidate hypotheses from logs, creating test cases, reviewing code, drafting runbooks, analyzing postmortem timelines and identifying recurring patterns. The important part is validation: AI output is a hypothesis or acceleration mechanism, not an authoritative production decision.

Q. How would you use AI during a major incident?

Spoken answer: I would provide the AI with approved, appropriately scoped telemetry and ask it to correlate symptoms, recent changes, known incidents and relevant runbooks. It can generate candidate causes and investigation steps. I would validate each recommendation against actual metrics, logs and system behavior before executing anything. For high-risk actions, I would require explicit human approval and follow existing change and incident controls.

Q. What guardrails would you establish for enterprise AI use?

Spoken answer: I would define approved tools and data classifications, prohibit sensitive data from unauthorized systems, enforce least privilege and maintain auditability. Prompts and outputs involving operational decisions should be treated according to the sensitivity of the underlying data. I would require human validation for production changes, security decisions and customer-impacting remediation. I would also maintain reusable prompts and evaluation cases so teams can measure whether AI workflows are actually reliable.

Q. How do you evaluate an AI-generated operational recommendation?

Spoken answer: I ask: what evidence supports it, what assumptions were made, what could be wrong, and what is the blast radius if we act on it? I cross-check against monitoring data, logs, configuration and recent changes. I prefer a reversible diagnostic or mitigation step first. For recurring workflows, I create test cases and measure precision, false positives and failure modes before allowing broader adoption.

Q. How would you decompose an ambiguous problem for an AI agent?

Spoken answer: I first define the desired outcome, constraints, available evidence and definition of done. Then I split the work into bounded tasks—for example collect telemetry, identify candidate failure modes, inspect configuration, generate a test plan and summarize evidence. Each task gets explicit inputs and validation criteria. The final integration remains human-controlled, especially where security, reliability or production state is involved.

9. Security / Risk / Enterprise

Q. How do security and reliability intersect?

Spoken answer: Security failures can directly become reliability failures through compromised credentials, resource exhaustion, malicious traffic or unsafe changes. I therefore include identity, least privilege, secrets management, network controls, dependency risk and auditability in reliability design. Conversely, reliability controls such as rate limiting, isolation and graceful degradation can improve security resilience. I treat security and operational risk as design constraints rather than a final review step.

Q. How do you handle secrets in CI/CD and Kubernetes?

Spoken answer: I avoid hard-coding secrets in repositories or images. Secrets should come from an approved secret-management mechanism, with least-privilege access and rotation. CI/CD identities should have only the permissions needed for their deployment scope. I also ensure logs do not expose credentials and that secret access is auditable. For Kubernetes, I distinguish between Kubernetes Secret objects and a properly managed external secret system depending on the enterprise architecture.

Q. How do you manage change risk in production?

Spoken answer: I use automated validation, peer review, small changes, staged rollout and rollback plans. For high-risk changes I prefer canary or progressive delivery and explicit health criteria. Observability must be available before the change. I also correlate incidents with recent changes because deployment events are often a strong signal during triage. The objective is not zero change; it is controlled change with measurable risk.

10. Scenario / Behavioral Questions

Q. Tell me about a difficult production problem you solved.

Spoken answer: In my recent SRE work, I've handled customer-impacting issues where the first symptom was not the root cause. My approach is consistent: establish impact, inspect service and platform telemetry, compare against recent changes, isolate the failing component, mitigate safely, then complete RCA. I focus particularly on improving the system after recovery—alert quality, automation, runbooks or validation—so the operational burden does not remain with the on-call team.

Q. Tell me about a time you automated manual work.

Spoken answer: I look for repeated operational tasks such as health checks, log analysis, deployment validation or remediation. I first document the manual decision process, identify what can be deterministic, then implement automation with validation, logging, error handling and safe rollback behavior. I measure the result in terms of engineer time saved, execution consistency and reduction in recurrence. I prefer automation that becomes a reusable platform capability rather than a one-off script.

Q. What would you do in your first 30/60/90 days?

Spoken answer: First 30 days: understand the services, architecture, SLOs, incident history, operational model and team interfaces. I would identify the highest reliability and toil hotspots. By 60 days, I would take ownership of selected services, improve one or two concrete reliability gaps and contribute to incident response and design reviews. By 90 days, I would aim to have measurable improvements—such as reduced alert noise, reduced toil, stronger SLO telemetry, improved recovery automation or better operational readiness—and a roadmap for larger reliability work.

Q. Why should we hire you for a Lead SRE position?

Spoken answer: I bring a combination of hands-on production operations and reliability engineering. I have worked across Azure, AWS, Kubernetes, Terraform, CI/CD, observability and incident management, and I'm comfortable moving from an incident symptom into the underlying system design. I also think in terms of automation, SLOs and operational ownership rather than just ticket closure. For a lead role, I can contribute technically while helping the team turn ambiguous reliability problems into structured, measurable engineering work.

11. Rapid-Fire Technical Questions

Q. What are the four golden signals?

Spoken answer: Latency, traffic, errors and saturation. I use them as a starting point for service health, then add domain-specific indicators.

Q. What is a burn rate?

Spoken answer: Burn rate describes how quickly a service is consuming its allowed error budget relative to the SLO window. High burn rate means the service is trending toward violating its objective quickly.

Q. What is idempotency?

Spoken answer: An operation is idempotent when repeating it produces the same intended final state. It is essential for safe retries and reconciliation.

Q. What is a circuit breaker?

Spoken answer: A control that stops repeated calls to a failing dependency for a period, allowing recovery and preventing cascading failure.

Q. What is graceful degradation?

Spoken answer: Maintaining core functionality while reducing or disabling non-critical functionality during resource or dependency failures.

Q. What is a blameless postmortem?

Spoken answer: A factual analysis of system conditions, decisions and contributing factors without focusing on individual blame; the goal is learning and prevention.

Q. What is an SRE error budget policy?

Spoken answer: A documented rule for how teams respond when reliability budget is being consumed too quickly—for example prioritizing reliability work or restricting risky changes.

Q. What is high cardinality in metrics?

Spoken answer: A metric with too many unique label combinations, which can increase storage and query cost and degrade the monitoring system itself.

Q. What is a Kubernetes readiness probe?

Spoken answer: It indicates whether a pod should receive traffic. A failed readiness check removes the pod from service endpoints without necessarily restarting it.

Q. What is a liveness probe?

Spoken answer: It indicates whether a container is alive enough to continue running. Repeated failure can cause Kubernetes to restart the container.

Q. What is a Kubernetes HPA?

Spoken answer: Horizontal Pod Autoscaler adjusts workload replica count based on configured metrics such as CPU, memory or custom/external metrics.

Q. What is Terraform drift?

Spoken answer: A difference between the state Terraform expects and the actual infrastructure. Drift can be detected through planning and corrected according to the intended ownership model.

Final Interview Checklist

Be ready to demonstrate:

- A real P1/P2 incident: symptoms, telemetry, mitigation, RCA and prevention.
- A real automation example with measurable toil reduction.
- A Python coding example: API/automation/data processing plus tests and error handling.
- SLI/SLO design for a service you know well.
- Prometheus/Grafana alert and dashboard design.
- Kubernetes troubleshooting from pod → events → logs → resources → networking → dependencies.
- Terraform state/drift and ownership boundaries.
- A declarative reconciliation/self-healing design.
- An AI-assisted SRE workflow with explicit validation and data-security controls.
- How you lead engineers through ambiguity without becoming a bottleneck.

Likely Lead-Level Follow-Ups

Expect interviewers to push beyond definitions with questions such as: *“What metric proves that?” “What happens if your automation is wrong?” “How would you test this?” “What is the blast radius?” “How do you know the SLO is meaningful?” “What would you automate versus keep human-controlled?” “How would this work at enterprise scale?”*

Important: Keep answers grounded in your actual experience. The strongest answers connect a technical decision to customer impact, reliability, risk, cost and measurable outcomes.